API Gateway et microservices sécurisés avec Docker Compose et GitLab

Entreprise fictive : NOVA SERVICES· Niveau débutant / intermédiaire · Durée estimée : 2 à 3 heures

Ce TP reprend le contexte de notre entreprise fictive. Cette fois, nous allons construire une architecture de microservices avec FastAPI, une API Gateway, Docker Compose et une pipeline GitLab CI/CD, en intégrant de bonnes pratiques de sécurité DEVSECOPS.

1. Contexte de l'entreprise

NOVA SERVICES voudrais creér une boutique en ligne qui expose plusieurs fonctionnalités indépendantes :

- Users Service : gère les utilisateurs.
- Orders Service : gère les commandes.
- API Gateway : constitue le point d'entrée unique des clients.
- GitLab CI/CD : automatise les tests et la construction des images Docker.
- GitLab Container Registry : stocke les images Docker produites par la pipeline.

L'objectif est qu'un client puisse obtenir les informations d'un utilisateur et ses commandes à travers une seule requête, sans connaître l'organisation interne des microservices.

Client web / mobile

Appels HTTP vers localhost:3000

API Gateway — FastAPI

Routage · Validation · Authentification · Agrégation BFF

Users Service Port interne 8000

Orders Service Port interne 8000

GitLab CI/CD + Container Registry

Tests · Build · Publication des images

2. Objectifs pédagogiques

À la fin du TP, tu devras être capable de :

1. Créer deux microservices indépendants avec FastAPI.

2. Construire une API Gateway avec des routes `/users`, `/orders` et `/profile/{id}`.
3. Mettre en œuvre le pattern Backend For Frontend (BFF) pour agréger plusieurs réponses.
4. Dockeriser chaque application.
5. Déployer l'ensemble avec Docker Compose sur un réseau privé.
6. Ajouter des contrôles de sécurité : authentification, validation, timeouts, gestion des erreurs et réduction de l'exposition réseau.
7. Créer une pipeline GitLab CI/CD qui teste et construit les images.
8. Publier les images dans le GitLab Container Registry.

3. Architecture du projet

Nous allons utiliser cette structure de répertoires :

```
api/
├── users/
│   ├── main.py
│   ├── requirements.txt
│   └── Dockerfile
├── orders/
│   ├── main.py
│   ├── requirements.txt
│   └── Dockerfile
├── gateway/
│   ├── main.py
│   ├── requirements.txt
│   └── Dockerfile
├── tests/
│   └── test_gateway.py
├── docker-compose.yml
├── .dockerignore
├── .gitignore
└── .gitlab-ci.yml
```

Principe de sécurité : seul le service Gateway sera publié sur le port `3000` de la machine. Les services Users et Orders resteront accessibles uniquement sur le réseau Docker interne.

Partie 1 — Développer les microservices

4. Étape 1 : créer le service Users

Ce service expose une liste d'utilisateurs. Dans ce TP, les données sont fictives et stockées en mémoire : il n'y a pas encore de base de données.

Crée `users/main.py` :

```
from fastapi import FastAPI

app = FastAPI(title="E-Shop Users Service")

users = [
    {"id": 1, "name": "Alice", "email": "alice@test.com"},
    {"id": 2, "name": "Bob", "email": "bob@test.com"},
]


@app.get("/health")
def health():
    return {"status": "ok"}


@app.get("/users")
def get_users():
    return users


@app.get("/users/{user_id}")
def get_user(user_id: int):
    user = next((u for u in users if u["id"] == user_id), None)

    if user is None:
        from fastapi import HTTPException
        raise HTTPException(status_code=404, detail="User not found")

    return user
```

Crée `users/requirements.txt` :

```
fastapi
uvicorn
```

Dockerfile du service Users

Crée `users/Dockerfile` :

```
```

Bonne pratique sécurité : l'application tourne avec un utilisateur non privilégié plutôt qu'en `root`. L'image de base est légère et les dépendances sont installées sans conserver le cache pip.

5. Étape 2 : créer le service Orders

Crée `orders/main.py` :

```
from fastapi import FastAPI, HTTPException

app = FastAPI(title="E-Shop Orders Service")

orders = [
    {"id": 101, "userId": 1, "item": "Laptop"},
    {"id": 102, "userId": 2, "item": "Phone"},
    {"id": 103, "userId": 1, "item": "Keyboard"},
]


@app.get("/health")
def health():
    return {"status": "ok"}


@app.get("/orders")
def get_orders():
    return orders


@app.get("/orders/user/{user_id}")
def get_orders_by_user(user_id: int):
    return [order for order in orders if order["userId"] == user_id]


@app.get("/orders/{order_id}")
def get_order(order_id: int):
    order = next((o for o in orders if o["id"] == order_id), None)

    if order is None:
        raise HTTPException(status_code=404, detail="Order not found")

    return order
```

Crée `orders/requirements.txt` :

fastapi
uvicorn

Crée `orders/Dockerfile` :

6. Étape 3 : tester les microservices seuls

Avant d'ajouter la Gateway, vérifie que chaque application fonctionne. Depuis la racine du projet :

docker build -t eshop-users:local ./users
docker build -t eshop-orders:local ./orders

Lance temporairement Users :

docker run --rm -p 8001:8000 eshop-users:local

Dans un autre terminal :

curl http://localhost:8001/users
curl http://localhost:8001/users/1

Arrête le conteneur avec `Ctrl+C`, puis teste Orders :

docker run --rm -p 8002:8000 eshop-orders:local
curl http://localhost:8002/orders
curl http://localhost:8002/orders/user/1

Les ports `8001` et `8002` sont utilisés uniquement pour ces tests temporaires. Dans l'architecture finale, ces services ne seront pas publiés directement sur la machine hôte.

Partie 2 — Construire l'API Gateway sécurisée

7. Étape 4 : définir les responsabilités de la Gateway

La Gateway va exposer quatre routes principales :

Endpoint public	Fonction
`GET /users`	Transmet la requête au service Users
`GET /orders`	Transmet la requête au service Orders
`GET /profile/1`	Agrège les informations de l'utilisateur et ses commandes
`GET /health`	Vérifie que la Gateway fonctionne

Nous allons ajouter les protections suivantes :

- Authentification par jeton Bearer pour les routes métier.
- Secret fourni par variable d'environnement, pas écrit dans le code.
- Timeout pour éviter que la Gateway attende indéfiniment un microservice.
- Gestion contrôlée des erreurs internes, sans renvoyer les détails techniques aux clients.
- Accès aux services backend par le réseau Docker privé.
- Pas de publication directe des ports Users et Orders.

> Pour un exercice, un jeton Bearer statique permet de comprendre le principe. En production, il faut généralement utiliser un fournisseur d'identité et des jetons courts (par exemple OAuth 2.0/OIDC), avec rotation des secrets, autorisations par rôle et validation des jetons. HTTPS est indispensable dès que le trafic sort d'un environnement local maîtrisé.

8. Étape 5 : écrire le code de la Gateway

Crée `gateway/main.py` :

```
import os
import asyncio
import secrets

import httpx
from fastapi import FastAPI, Depends, Header, HTTPException

app = FastAPI(title="E-Shop API Gateway")
```

```
USERS_SERVICE = os.getenv("USERS_SERVICE", "http://users:8000")
ORDERS_SERVICE = os.getenv("ORDERS_SERVICE", "http://orders:8000")

# Le token vient d'une variable d'environnement.
GATEWAY_TOKEN = os.getenv("GATEWAY_TOKEN", "")


# Vérification de l'authentification
async def require_token(
    authorization: str | None = Header(default=None),
):
    if not GATEWAY_TOKEN:
        raise HTTPException(
            status_code=500,
            detail="Gateway authentication is not configured",
        )

    if authorization is None or not authorization.startswith("Bearer "):
        raise HTTPException(
            status_code=401,
            detail="Missing or invalid authentication token",
            headers={"WWW-Authenticate": "Bearer"},
        )

    token = authorization.removeprefix("Bearer ")

    if not token or not secrets.compare_digest(token, GATEWAY_TOKEN):
        raise HTTPException(
            status_code=401,
            detail="Invalid authentication token",
            headers={"WWW-Authenticate": "Bearer"},
        )


# Fonction commune pour appeler les microservices
async def fetch_json(url: str):
    try:
        async with httpx.AsyncClient(timeout=5.0) as client:
            response = await client.get(url)
            response.raise_for_status()
            return response.json()

    except (httpx.HTTPError, ValueError) as exc:
        raise HTTPException(
            status_code=502,
            detail="A backend service is unavailable",
        ) from exc


# Health check public
@app.get("/health")
async def health():
    return {"status": "ok"}
```

```
# Route protégée : Users
@app.get("/users", dependencies=[Depends(require_token)])
async def proxy_users():
    return await fetch_json(f"{USERS_SERVICE}/users")


# Route protégée : Orders
@app.get("/orders", dependencies=[Depends(require_token)])
async def proxy_orders():
    return await fetch_json(f"{ORDERS_SERVICE}/orders")


# Route protégée : agrégation User + Orders
@app.get("/profile/{user_id}", dependencies=[Depends(require_token)])
async def get_profile(user_id: int):

    users, orders = await asyncio.gather(
        fetch_json(f"{USERS_SERVICE}/users"),
        fetch_json(f"{ORDERS_SERVICE}/orders"),
    )

    user = next(
        (u for u in users if u["id"] == user_id),
        None,
    )

    if user is None:
        raise HTTPException(
            status_code=404,
            detail="User not found",
        )

    user_orders = [
        order for order in orders
        if order["userId"] == user_id
    ]

    return {
        "user": user,
        "orders": user_orders,
    }
```

Crée `gateway/requirements.txt` :

```
fastapi
uvicorn
httpx
```

Crée `gateway/Dockerfile` :

Comprendre le pattern BFF

Lorsqu'un client appelle `/profile/1`, la Gateway :

1. Récupère les utilisateurs depuis Users.
2. Récupère les commandes depuis Orders.
3. Recherche l'utilisateur dont l'identifiant est 1.
4. Filtre les commandes associées à cet utilisateur.
5. Renvoie une réponse combinée.

Les deux appels backend sont lancés en parallèle avec `asyncio.gather`, ce qui évite de les attendre successivement.

Exemple de réponse attendue :


```
{
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@test.com"
  },
  "orders": [
    {
      "id": 101,
      "userId": 1,
      "item": "Laptop"
    },
    {
      "id": 103,
      "userId": 1,
      "item": "Keyboard"
    }
  ]
}
```


Point de vigilance : cette version récupère les listes complètes pour réaliser l'agrégation, comme dans l'énoncé initial. Pour un vrai système, il vaut mieux prévoir des endpoints filtrés par identifiant, une pagination et des contrôles d'autorisation sur les données demandées.

Partie 3 — Déployer avec Docker Compose

9. Étape 6 : gérer la configuration et les secrets

Nous allons utiliser une variable d'environnement pour le jeton d'accès. Le secret ne doit pas être codé en dur dans `main.py`, dans le Dockerfile ou dans le dépôt Git.

Génère un jeton local :

openssl rand -hex 32

Crée un fichier `.env` à la racine du projet :

GATEWAY_TOKEN=COLLER_ICI_LE_JETON_GENERE

Remplace la valeur par le jeton généré.

Crée `.gitignore` :

```
.env
__pycache__/
.pytest_cache/
.venv/
*.pyc
```

Crée également `.dockerignore` :

```
.env
.git
.gitlab
__pycache__
.pytest_cache
.venv
*.pyc
```

Règle de sécurité : ne committe jamais `.env`. En production, utilise les variables protégées et masquées de GitLab CI/CD ou un gestionnaire de secrets adapté.

10. Étape 7 : créer le fichier Docker Compose

Crée `docker-compose.yml` à la racine :

Pourquoi cette configuration est-elle plus sécurisée ?

- Un seul point d'entrée publié : le port `3000` de la Gateway est lié à `127.0.0.1`. Les services Users et Orders n'ont aucun port publié.
- Réseau privé : le réseau `backend` est interne à Docker.
- Secrets externalisés : le jeton vient du fichier `.env`, absent du dépôt Git.
- Conteneurs durcis : les processus utilisent un utilisateur non-root, le système de fichiers est en lecture seule, les capacités Linux sont supprimées et l'escalade de privilèges est désactivée.
- Healthchecks : Compose attend que les microservices soient opérationnels avant de démarrer la Gateway.

Cette configuration convient à un environnement local de TP. Pour un déploiement en production, il faut notamment configurer HTTPS, les limites de ressources, les journaux, la supervision et une gestion centralisée des secrets.

11. Étape 8 : démarrer l'architecture complète

Vérifie d'abord que le fichier `.env` contient bien ton jeton.

Valide la configuration :

docker compose config

Construis les images et démarre les services :

docker compose up --build -d

Vérifie leur état :

docker compose ps
docker compose logs gateway

Les trois services doivent être en cours d'exécution, et leurs healthchecks doivent devenir `healthy`.

Pour arrêter l'application :

docker compose down

12. Étape 9 : tester les endpoints

Récupère le jeton dans ton fichier `.env`, puis exporte-le dans ton terminal. Avec un shell Bash ou Zsh :

```
set -a
source .env
set +a
```

Teste d'abord la Gateway sans authentification :

```
curl -i http://localhost:3000/users
```

Résultat attendu : `401 Unauthorized`.

C'est normal : les routes métier exigent un jeton Bearer.

Teste ensuite la liste des utilisateurs :

```
curl -i \
  -H "Authorization: Bearer $GATEWAY_TOKEN" \
  http://localhost:3000/users
```

Teste les commandes :

```
curl -i \
  -H "Authorization: Bearer $GATEWAY_TOKEN" \
  http://localhost:3000/orders
```

Teste l'agrégation BFF :

```
curl -i \
  -H "Authorization: Bearer $GATEWAY_TOKEN" \
  http://localhost:3000/profile/1
```

Teste un utilisateur inexistant :

```
curl -i \
  -H "Authorization: Bearer $GATEWAY_TOKEN" \
  http://localhost:3000/profile/999
```

Résultat attendu : `404 Not Found`.

Vérifie également le endpoint de santé :

```
curl -i http://localhost:3000/health
```

Celui-ci ne nécessite pas d'authentification, car il sert uniquement à vérifier que le processus Gateway répond. Il ne révèle pas d'informations détaillées sur les microservices.

Attention : l'authentification protège ici l'accès à l'API via la Gateway, mais les autorisations sont encore simplifiées. Dans une vraie application, il faudrait vérifier que l'utilisateur

authentifié a le droit de consulter le profil demandé, et pas seulement qu'il possède un jeton valide.

Partie 4 — Tests automatisés

13. Étape 10 : tester la Gateway avec pytest

Crée le fichier `tests/test_gateway.py` :

```
import os

from fastapi.testclient import TestClient

from gateway import main as gateway

TOKEN = os.getenv("GATEWAY_TOKEN", "ci-test-token")

gateway.GATEWAY_TOKEN = TOKEN

client = TestClient(gateway.app)


def test_health_is_public():
    response = client.get("/health")

    assert response.status_code == 200
    assert response.json() == {"status": "ok"}


def test_users_requires_authentication():
    response = client.get("/users")

    assert response.status_code == 401


def test_users_with_invalid_token():
    response = client.get(
        "/users",
        headers={"Authorization": "Bearer wrong-token"},
    )

    assert response.status_code == 401


def test_profile_aggregates_user_and_orders(monkeypatch):
    async def fake_fetch_json(url):
        if url.endswith("/users"):
            return [
```

```
                {"id": 1, "name": "Alice", "email": "alice@test.com"},
                {"id": 2, "name": "Bob", "email": "bob@test.com"},
            ]

        if url.endswith("/orders"):
            return [
                {"id": 101, "userId": 1, "item": "Laptop"},
                {"id": 102, "userId": 2, "item": "Phone"},
                {"id": 103, "userId": 1, "item": "Keyboard"},
            ]

        raise AssertionError(f"Unexpected URL: {url}")

    monkeypatch.setattr(gateway, "fetch_json", fake_fetch_json)

    response = client.get(
        "/profile/1",
        headers={"Authorization": f"Bearer {TOKEN}"},
    )

    assert response.status_code == 200

    data = response.json()

    assert data["user"]["id"] == 1
    assert [order["id"] for order in data["orders"]] == [101, 103]


def test_profile_unknown_user(monkeypatch):
    async def fake_fetch_json(url):
        if url.endswith("/users"):
            return [{"id": 1, "name": "Alice", "email": "alice@test.com"}]

        return [{"id": 101, "userId": 1, "item": "Laptop"}]

    monkeypatch.setattr(gateway, "fetch_json", fake_fetch_json)

    response = client.get(
        "/profile/999",
        headers={"Authorization": f"Bearer {TOKEN}"},
    )

    assert response.status_code == 404
```

Installe les dépendances et lance les tests depuis la racine du projet :

```
python -m pip install -r gateway/requirements.txt pytest
export GATEWAY_TOKEN=ci-test-token
python -m pytest -q
```

Les tests vérifient que la route de santé répond, que les routes métier sont protégées et que l'agrégation retourne les bonnes commandes.

Ces tests isolent la Gateway des microservices en remplaçant les appels réseau par des réponses simulées. Ils ne remplacent donc pas les tests d'intégration avec Docker Compose.

Partie 5 — Automatiser avec GitLab CI/CD

14. Étape 11 : comprendre la pipeline

Nous allons automatiser deux étapes principales :

1. Push du code sur GitLab

Déclenche la pipeline

2. Stage test

Installation des dépendances · pytest

3. Stage build & push

Construction des trois images Docker

4. GitLab Container Registry

Publication des images avec le SHA du commit

15. Étape 12 : créer `.gitlab-ci.yml`

À la racine du projet, crée le fichier `.gitlab-ci.yml` :

```
```

Points importants sur la pipeline

- Le stage `test` doit réussir avant le stage `build`.
- Les images sont publiées avec `$CI_COMMIT_SHA`, ce qui permet de retrouver le commit correspondant à chaque version.
- Le job utilise les variables prédéfinies de GitLab pour se connecter au Registry. Il n'est pas nécessaire d'inscrire un mot de passe personnel dans le fichier YAML.

- Le jeton déclaré dans le job de test est un jeton fictif réservé aux tests ; il ne doit jamais être réutilisé en production.
- Docker-in-Docker nécessite un runner GitLab configuré pour cette méthode, généralement avec le mode privilégié. Sur un runner qui ne l'autorise pas, il faudra utiliser une méthode de build adaptée à son environnement.

Pour les détails du Registry, consulte la documentation officielle : GitLab Container Registry.

16. Étape 13 : publier le projet sur GitLab

Depuis la racine du projet :

```
git init
git add .
git status
```

Avant de commit, vérifie que `.env` n'apparaît pas dans les fichiers à ajouter.

Puis :

```
git commit -m "Initial E-Shop API Gateway project"
git branch -M main
git remote add origin URL_DE_TON_DEPOT_GITLAB
git push -u origin main
```

Remplace `URL_DE_TON_DEPOT_GITLAB` par l'URL réelle de ton dépôt.

Dans GitLab :

1. Ouvre le projet.
2. Va dans Build > Pipelines.
3. Vérifie que les stages `test` et `build` s'exécutent.
4. Consulte le journal d'un job en échec.
5. Après une pipeline réussie, ouvre Deploy > Container Registry (le libellé peut varier selon la version de GitLab).
6. Vérifie que les images `users`, `orders` et `gateway` ont été publiées.

Les noms d'images suivront cette forme :

```
registry.gitlab.com/groupe/projet/users:<commit-sha>
registry.gitlab.com/groupe/projet/orders:<commit-sha>
registry.gitlab.com/groupe/projet/gateway:<commit-sha>
```

Les valeurs `groupe`, `projet` et `<commit-sha>` seront remplacées par celles de ton dépôt.

Améliorations possibles de la pipeline (optionnel)

Pour une pipeline plus robuste, ajoute ensuite :

- un contrôle de vulnérabilités des dépendances Python ;
- un scan de sécurité des images Docker ;
- une analyse statique du code ;
- des tests d'intégration ;
- une étape de déploiement manuel vers un environnement de test ;
- une stratégie de promotion des mêmes images vers la production, sans reconstruire des images différentes.

Partie 6 — Bonnes pratiques de sécurité

17. Checklist de sécurité

Validation de la sécurité

0/10

Le fichier .env est exclu de Git.
La Gateway exige un jeton sur les endpoints métier.
Les services Users et Orders ne publient aucun port sur la machine hôte.
Les communications internes passent par un réseau Docker dédié.
Les conteneurs tournent avec un utilisateur non-root.
Les requêtes vers les microservices ont un timeout.
Les erreurs backend ne révèlent pas les détails techniques au client.
La pipeline utilise les variables prédéfinies du Registry GitLab.
Les images sont taguées avec le SHA du commit.
Les images et dépendances sont analysées avant un déploiement de production.

Les points à retenir :

- Authentification ≠ autorisation. Vérifier qu'un jeton existe ne signifie pas qu'un client peut consulter toutes les commandes ou tous les profils.
- Un réseau privé n'est pas une authentification. Les microservices doivent rester protégés même si le trafic est interne.
- Un secret statique est adapté à un exercice, pas à une architecture de production. Utilise une solution d'identité adaptée et prévois la rotation des secrets.
- Ne publie pas les ports backend inutilement. Les clients doivent passer par la Gateway.
- Ne considère pas une image construite comme une image sécurisée. Il faut aussi contrôler les dépendances, les images de base et les vulnérabilités.

18. Questions de compréhension

Question 1 — Quels avantages l'API Gateway apporte-t-elle ?

Réponse attendue : elle fournit un point d'entrée unique, centralise certaines règles de sécurité et simplifie les appels du client. Elle peut également agréger les données de plusieurs microservices afin de réduire le nombre de requêtes nécessaires côté client.

Question 2 — Quels sont les inconvénients d'une API Gateway ?

Réponse attendue :

- Elle peut devenir un point de défaillance si elle n'est pas redondée.
- Elle ajoute une étape réseau et donc une latence potentielle.
- Elle peut devenir trop complexe si toutes les responsabilités y sont regroupées.
- Elle nécessite une supervision, une gestion des secrets et une mise à jour régulière.

Question 3 — Comment gérer l'authentification et la sécurité ?

Réponse attendue : valider les jetons d'accès à la Gateway, appliquer les autorisations nécessaires à chaque ressource, utiliser HTTPS, protéger les secrets, limiter le débit des requêtes et contrôler les accès aux services internes. Les microservices doivent également vérifier les autorisations sensibles lorsqu'elles s'appliquent à leurs données.

Question 4 — Quelle différence entre une API Gateway et un reverse proxy NGINX ?

Réponse attendue : un reverse proxy comme NGINX peut router les requêtes et gérer TLS, les en-têtes et les règles de trafic. Une API Gateway est généralement conçue pour la gestion des API et peut proposer des fonctions supplémentaires telles que l'authentification, les quotas, la gestion des clients et l'agrégation BFF. Les deux catégories peuvent se recouper : NGINX peut aussi être configuré pour remplir certaines fonctions d'API Gateway.

19. Livrables à rendre

À la fin du TP, tu dois fournir :

- Les trois applications FastAPI : Users, Orders et Gateway.
- Les trois Dockerfiles.
- Le fichier `docker-compose.yml`.
- Le fichier `.gitlab-ci.yml`.
- Les tests automatisés `pytest`.
- Des captures ou sorties de commandes montrant les réponses des trois endpoints.
- Une capture de la pipeline GitLab réussie.
- Une capture du Container Registry avec les trois images publiées.
- Des réponses rédigées aux quatre questions de compréhension.